ESPECIFICACIÓN DE ARQUITECTURA BACKEND v2.0

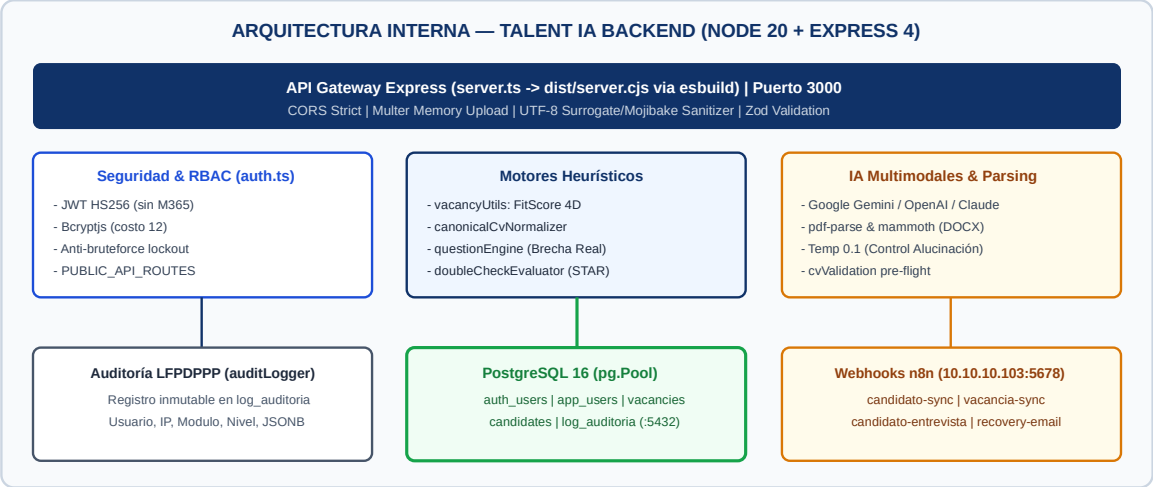
ARQUITECTURA TÉCNICA: TALENT IA BACKEND

API REST Node.js 20 LTS + Express 4 + PostgreSQL + Motores IA & n8n

Repositorio:	eaguilarh01/ talent_ia_backend.git	**Autor:**	Enrique Aguilar
Organización:	EAS Consulting Group	**Fecha:**	Noviembre 2026
Compilación:	esbuild (dist/server.cjs)	**Contenedor:**	eas-recruiting-backend (:3000)

1. Visión General y Arquitectura Interna

El repositorio **talent_ia_backend** implementa la API REST, el motor determinista de evaluación curricular (FitScore 4D y Double-Check), la integración multimodal con LLMs (Google Gemini, OpenAI, Anthropic Claude), el despachador de webhooks n8n y la persistencia relacional transaccional en PostgreSQL. Se compila mediante `build-server.js` (esbuild) generando el bundle CommonJS `dist/server.cjs`.



2. Stack Tecnológico del Backend

Componente	Tecnología	Versión	Propósito
Runtime y Web	Node.js 20 + Express 4	20-slim / 4.21	API REST asíncrona con sanitización UTF-8 estricta
Compilador	esbuild + TypeScript 5.8	0.25 / 5.8.2	Resolución NodeNext y bundling a dist/server.cjs
Base de Datos	PostgreSQL (pg.Pool)	^8.22.0	Pooling nativo y consultas SQL 100% parametrizadas
Motores IA	Gemini / OpenAI / Claude	2.4 / 7.5 / 0.120	Extracción semántica de CVs con temperatura controlada
Seguridad	bcryptjs + jsonwebtoken	3.0.3 / 9.0.3	Hashing costo 12 y firma JWT HS256 (sin M365)
Parsing y Tests	pdf-parse / mammoth / tsx	2.4 / 1.12 / 4.21	Lectura de CVs en memoria y 7 suites unitarias

Componente	Tecnología	Versión	Propósito

3. Modelo Relacional PostgreSQL y Seguridad

3.1 Tablas Principales (database_schema.sql & db/schema.sql)

Tabla	Columnas Clave	Uso y Reglas de Integridad
`auth_users`	id UUID, email UNIQUE, password_hash, reset_code	Credenciales seguras bcrypt; trigger on_auth_user_created
`app_users`	id FK, nombre, rol, activo, primer_login	Roles permitidos: admin, leader, user, director
`vacancies`	id, title, department, min_competencies JSONB	Requisitos, años de experiencia, rango salarial e inglés
`candidates`	id, resume_text, cv_base_score, double_check_delta, manual_feedback_delta, fit_score_breakdown, evaluation_notes	Invariante clamp(cvBase + dcDelta + manualDelta, 0, 100); sanitización UTF-8
`log_auditoria`	usuario_email, ip_address, accion, detalles JSONB	Bitácora inmutable LFPDPPP indexada por fecha DESC

3.2 Seguridad, Pipeline Unificado CV y Evaluación Descompuesta

- **Soberanía JWT y Cero Secretos:** Lista blanca explícita `PUBLIC_API_ROUTES`; bloqueo anti-fuerza bruta, PIN OTP de 30 min vía n8n y cuentas bootstrap solo mediante `.env` (>= 12 caracteres).
- **Pipeline Unificado CV (`resolveAndNormalizeCandidateCvProfile + detectDegradedCvFields`):** Aplicado en ingestión nueva (`/analyze-candidate`, `/public/applications`, `/candidates`) y migración histórica (`recalibrateAllCandidateScores` y `POST /api/candidates/rehydrate-all`).
- **Motores Coherentes y Notas Reversibles:** `auditCandidateRequirements` sirve como fuente única para `vacancyUtils.ts` y `questionEngine.ts` (ponderación por secciones, umbral >= 60%); `doubleCheckEvaluator.ts` soporta calibración bidireccional IA [-25, +15]; `/api/submit-feedback` y `POST /api/candidates/:id/notes/:noteId/revert` gestionan `evaluation_notes[]` sin perder deltas tras reinicios.

4. Webhooks n8n y Despliegue Docker

El archivo `n8n_workflow_eas_recruiting_master.json` orquesta 4 flujos asíncronos (`candidato-sync`, `vacancia-sync`, `candidato-entrevista` y `recovery-email`). La orquestación en producción se define en `docker-compose.yml` sobre la red externa `atlas-net`.

```
npm install

npm run lint     # Verificación TypeScript NodeNext
```

```
npm test         # Ejecución de las 7 suites de pruebas (auth, cvValidation, scoring, normalizer)

npm run build    # Genera dist/server.cjs con esbuild

docker compose up -d --build
```